Python语言与系统设计

（字符串与正则表达式）

Python Language & System Design

第4章 字符串与正则表达式

(String & Regular Expression)

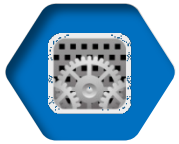
合肥工业大学 计算机与信息学院



第4章 字符串与正则表达式


4.1 字符串


4.2 字符串的应用


4.3 正则表达式


4.4 正则表达式的应用


4.5 本章小结

4.1 字符串

4.1.1 字符串概述

- 最早的字符串编码是美国标准信息交换码ASCII，仅对10个数字、26个大写英文字母、26个小写英文字母及一些其他符号进行了编码。ASCII码采用1个字节来对字符进行编码，最多只能表示256个符号；
- 多种不同的编码格式，常见的主要有UTF-8、UTF-16、UTF-32、GB2312、GBK、CP936、base64、CP437等；
- GB2312是我国制定的中文编码，使用1个字节表示英语，2个字节表示中文；GBK是GB2312的扩充，而CP936是微软在GBK基础上开发的编码方式。GB2312、GBK和CP936都是使用2个字节表示中文；
- UTF-8对全世界所有国家需要用到的字符进行了编码，以1个字节表示英语字符(兼容ASCII)，以3个字节表示常见汉字，还有些语言的符号使用2个字节（例如俄语和希腊语符号）或4个字节。

注：slides参考：董付国. 《Python程序设计》(第2版). 清华大学出版社, 2018.

4.1 字符串

- Python 3.x完全支持中文字符，默认使用UTF8编码格式，无论是一个数字、英文字母，还是一个汉字，在统计字符串长度时都按一个字符对待和处理。

```
>>> s = '合肥工业大学'
>>> len(s)                          #字符串长度，或者包含的字符个数
6

>>> s = '合肥工业大学HFUT'     #中文与英文字符同样对待，都算一个字符
>>> len(s)
10

>>> 姓名 = '张三'                    #使用中文作为变量名
>>> print(姓名)                      #输出变量的值
张三
```

4.1 字符串

4.1.1 字符串概述

- 在Python中，字符串属于不可变序列类型，除了支持序列通用方法（包括分片操作）以外，还支持特有的字符串操作方法。

```
>>> test='good'
>>> test[1]='a‘
异常
```

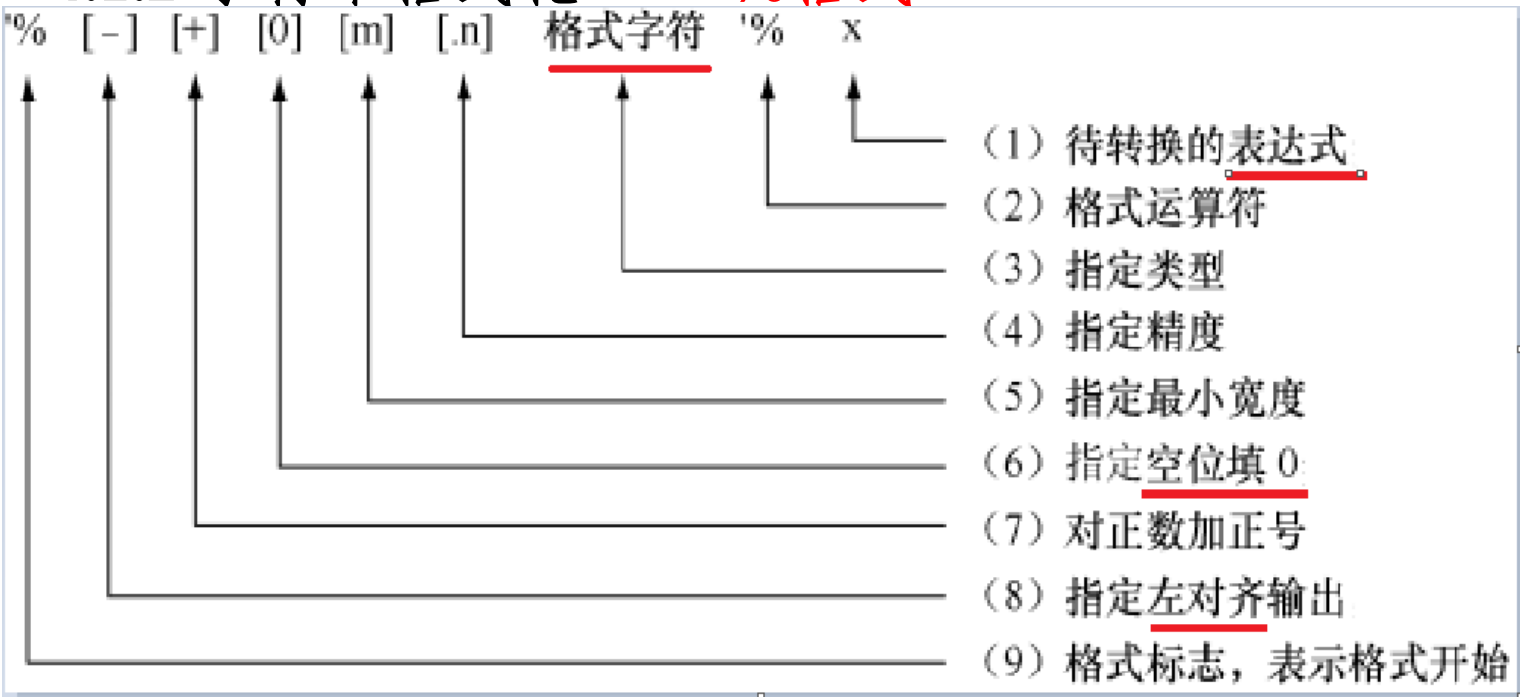
- Python字符串驻留机制：对于短字符串，将其赋值给多个不同的对象时，内存中只有一个副本，多个对象共享该副本。长字符串不遵守驻留机制。

```
>>> a='good'
>>> b='good'
>>> id(a)==id(b)
True
```

```
>>> a='good'*20000
>>> b='good'*20000
>>> id(a)==id(b)
False
```

4.1 字符串

4.1.2 字符串格式化 --- %格式




```
>>> print('%5.2f'%3.14159)
 3.14
>>> print('%10.2f'%3.14159)
      3.14
>>> print('%-10.2f'%3.14159)
3.14      |
>>> print('%+010.2f'%3.14159)
+000003.14
```

%后跟表达式

默认右对齐

- 左对齐

空位填0

4.1 字符串

➢ 格式字符

格式字符	说明
%s	字符串 (采用str()的显示)
%r	字符串 (采用repr()的显示)
%c	单个字符
%d	十进制整数
%i	十进制整数
%o	八进制整数
%x	十六进制整数
%e	指数 (基底写为e)
%E	指数 (基底写为E)
%f、%F	浮点数
%g	指数(e)或浮点数 (根据显示长度)
%G	指数(E)或浮点数 (根据显示长度)
%%	一个字符"%"

4.1.2 字符串格式化-%格式

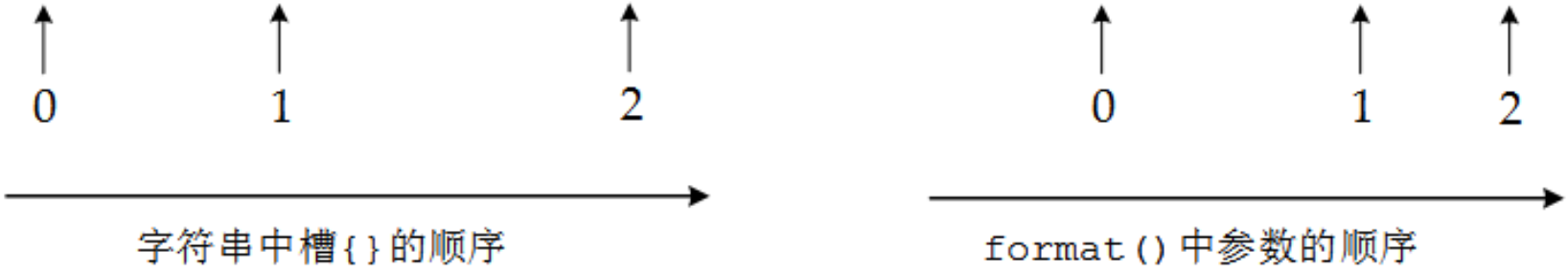
```
>>> x = 1235
>>> so = "%o" % x
>>> so
"2323"
>>> sh = "%x" % x
>>> sh
"4d3"
>>> se = "%e" % x
>>> se
"1.235000e+03"
>>> chr(ord("3")+1)
"4"
>>> "%s" % 65333
"65333"
>>> "%d" % "555"
TypeError: %d format: a number is required, not str
>>> '%s'%[1, 2, 3]          #直接把对象转换成字符串
'[1, 2, 3]'
```

4.1.2 字符串格式化

- 新格式化方法---format()

format()方法的格式： **<模板字符串>.format(<逗号分隔的参数>)**

```
"{ }：计算机{ }的CPU占用率为{ }%。".format("2016-12-31","PYTHON",10)
```




```
"{1}：计算机{0}的CPU占用率为{2}%。".format("2016-12-31","PYTHON",10)
```

4.1.2 字符串格式化

- format()方法中模板字符串的槽除了包括参数序号，还可以包括格式控制信息。
  - **槽的内部样式：{<参数序号>: <格式控制标记>}**
- 格式控制标记用来控制参数显示时的格式：
  - **<填充><对齐><宽度>,<.精度><类型>6个字段**

:	<填充>	<对齐>	<宽度>	,	<.精度>	<类型>
引导符号	用于填充的单个字符	< 左对齐 > 右对齐 ^ 居中对齐	槽的设定输出宽度	数字的千位分隔符 适用于整数和浮点数	浮点数小数部分的精度 或 字符串的最大输出长度	整数类型 b, c, d, o, x, X 浮点数类型 e, E, f, %

```
>>> "{0:*<8.2f}---{1:~>36.5}".format(3.14,"do-you-like-python")
'3.14****---                                do-yo'
```

str.format()

基本语法是通过 {} 和 : 来代替以前的 % 。

format 函数可以接受不限个参数，位置可以不按顺序。

```
>>>"{} {}".format("hello", "world") # 不设置指定位置，按默认顺序
'hello world'
>>> "{0} {1}".format("hello", "world") # 设置指定位置
'hello world'
>>> "{1} {0} {1}".format("hello", "world") # 设置指定位置
'world hello world'
```

数字格式化：

```
>>> print("{:.2f}".format(3.1415926));
3.14
```

- 超过约定槽位宽度。挤爆，无约束作用

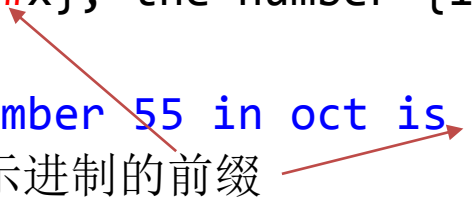
```
>>> "{:x<4d}".format(10)
'10xx'
>>> "{:x<4d}".format(12345)
'12345'
```

- 在剩余槽宽不对称，占位符如何布局？？

```
>>> "{:*^11d}".format(124)
'****124****'
>>> "{:*^11d}".format(1234)
'***1234****'
```

- 使用format方法进行格式化

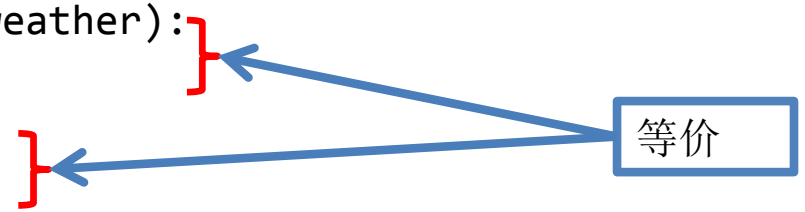
```
>>> print("The number {0:,} in hex is: {0:#x}, the number {1} in
   oct is {1:#o}".format(5555,55))
The number 5,555 in hex is: 0x15b3, the number 55 in oct is 0o67
```




```
>>> print("The number {0:,} in hex is: {0:x}, the number {1} in oct
   is {1:o}".format(5555,55))
The number 5,555 in hex is: 15b3, the number 55 in oct is 67

>>> '{0:<8d},{0:^8d},{0:>8d}'.format(65) #设置对齐方式
'65      ,   65   ,      65'
```

```
weather = [("Monday","rainy"),("Tuesday","sunny"),
           ("Wednesday", "sunny"),("Thursday","rainy"),
           ("Friday","cloudy")]
formatter = "Weather of '{0[0]}' is '{0[1]}'".format
for item in map(formatter,weather):
    print(item)
for item in weather:
    print(formatter(item))
```



运行结果：

```
Weather of 'Monday' is 'rainy'
Weather of 'Tuesday' is 'sunny'
Weather of 'Wednesday' is 'sunny'
Weather of 'Thursday' is 'rainy'
Weather of 'Friday' is 'cloudy'
```

4.1 字符串

- 从Python 3.6.x开始支持一种新的字符串格式化方式，官方叫做Formatted String Literals，在字符串前加字母f，称之为f-string， 含义与字符串对象format()方法类似。

  它的占位符{}内包含变量，就会以对应的变量直接替换：

```
>>> name = 'Python'
>>> age = 1
>>> f'My name is {name}, and I am {age} years old.'
'My name is Python, and I am 1 years old.'
>>> pi=3.14159
>>> f'值是{pi:10.2f}'
'值是       3.14'
```

- 例：

```
>>> a,b = 3.2212, 5.34565
>>> print("两数之和为:%5.2f"%(a+b))
两数之和为: 8.57
>>> print("两数之和为:{0:5.2f}".format(a+b))
两数之和为: 8.57
>>> print("两数{0:f}与{1:f}之和为:{2:5.2f}".format(a,b,a+b))
两数3.221200与5.345650之和为: 8.57
>>> print(f"两数{a:f}与{b:f}之和为:{a+b:5.2f}")
两数3.221200与5.345650之和为: 8.57
```

4.1.3 字符串常用方法

操作	含义
+	连接
*	重复
<string>[ ] , index(), rindex()	索引
<string>[:]	剪切
len(<string>)	长度
upper(), lower()	字符串中字母大/小写
capitalize()、title()、swapcase()	首字母大写，每个单词首字母大写，大小写交换
strip()	去两边空格及去指定字符
split(), rsplit() , lsplit()	按指定字符分割字符串为数组
join()	连接两个字符串序列
find(), rfind()	搜索指定字符串
replace()	字符串替换
for <var> in <string>	字符串迭代
startswith(),endswith() center()、ljust()、rjust() zfill() isalnum() 等	字符串是否以指定字符串开始或结束 居中、左对齐或右对齐 返回指定宽度的字符串，在左侧以字符0进行填充 字符串是否为数字、字母

4.1.3 字符串常用方法

- find()、rfind()、index()、rindex()、count()

✓ find()和rfind方法分别用来查找一个字符串在另一个字符串指定范围（默认是整个字符串）中首次和最后一次出现的位置，如果不存在则返回-1；

✓ index()和rindex()方法用来返回一个字符串在另一个字符串指定范围中首次和最后一次出现的位置，如果不存在则抛出异常；

✓ count()方法用来返回一个字符串在当前字符串中出现的次数。

```
>>> s="apple,peach,banana,peach,pear"
>>> s.find("peach")
6
>>> s.find("peach",7)
19
>>> s.rfind("peach")
19
>>> s.count("peach")
2
>>> s.index('abc')
异常
```

✓ 应用：查找字符串中每个字符的第一次出现：

```
text = '''
东边来个小朋友叫小松，手里拿着一捆葱。
西边来个小朋友叫小丛，手里拿着小闹钟。
小松手里葱捆得松，掉在地上一些葱。
小丛忙放闹钟去拾葱，帮助小松捆紧葱.
小松夸小丛像雷锋，小丛说小松爱劳动。
'''

for index, ch in enumerate(text):
    if index == text.index(ch):
        print((index, ch), end= '')
```

index() 返回一个字符串在另一个字符串指定范围中首次出现的位置

(0, '\n')(1, '东')(2, '边')(3, '来')(4, '个')(5, '小')(6, '朋')(7, '友')(8, '叫')(10, '松')(11, '，')(12, '手')(13, '里')(14, '拿')(15, '着')(16, '一')(17, '捆')(18, '葱')(19, '。')(21, '西')(30, '丛')(37, '闹')(38, '钟')(47, '得')(50, '掉')(51, '在')(52, '地')(53, '上')(55, '些')(61, '忙')(62, '放')(65, '去')(66, '拾')(69, '帮')(70, '助')(74, '紧')(76, '.')(80, '夸')(83, '像')(84, '雷')(85, '锋')(89, '说')(92, '爱')(93, '劳')(94, '动')

4.1.3 字符串常用方法

- split()、rsplit()、partition()、rpartition()

✓ **split()和rsplit()**方法分别用来以指定字符为分隔符，把当前字符串从左往右或从右往左分隔成多个字符串，并返回包含分隔结果的列表；

✓ **partition()和rpartition()**用来以指定字符串为分隔符将原字符串分隔为**3**部分，即分隔符前的字符串、分隔符字符串、分隔符后的字符串，如果指定的分隔符不在原字符串中，则返回原字符串和两个空字符串。

```
>>> s = "apple,peach,banana,pear"
>>> s.split(",")
["apple", "peach", "banana", "pear"]
>>> s.partition(',')
('apple', ',', 'peach,banana,pear')
>>> s.rpartition(',')
('apple,peach,banana', ',', 'pear')
>>> s.rpartition('banana')
('apple,peach,', 'banana', ',pear')
>>> s = "2020-5-20"
>>> t = s.split("-")
>>> print(t)
['2020', '5', '20']
```

分隔符

- 对于split()和rsplit()方法，如果不指定分隔符，则字符串中的任何空白符号（空格、换行符、制表符等）都将被认为是分隔符，把连续多个空白字符看作一个分隔符。

```
>>> s = '\n\nhello\t\t world \n\n\n My name\t is Python   '
>>> s.split()
['hello', 'world', 'My', 'name', 'is', 'Python']
```

- split()和rsplit()方法还允许指定最大分割次数。

```
>>> s = '\n\nhello\t\t world \n\n\n My name is Python    '
>>> s.split(None, 1)
['hello', 'world \n\n\n My name is Python    ']
>>> s.rsplit(None, 1)
['\n\nhello\t\t world \n\n\n My name is', 'Python']
>>> s.split(None, 2)
['hello', 'world', 'My name is Python    ']
>>> s.rsplit(None, 2)
['\n\nhello\t\t world \n\n\n My name', 'is', 'Python']
```

4.1 字符串

- 字符串连接join()

```
>>> li = ["apple", "peach", "banana", "pear"]
>>> ','.join(li)
'apple,peach,banana,pear'
>>> '::'.join(li)
'apple::peach::banana::pear'
```

连接符

```
>>> li = ["apple", "peach", "banana", "pear"]
>>> ''.join(li)
'applepeachbananapear'
```

- **应用：**将字符串重复指定次数，并使用指定的分隔符进行连接，生成的字符串结尾处不带分隔符。

✓ 例如，concat('good', 5, ',')的返回结果为'good,good,good,good,good'

```
def concat(s, n, separator):
    return separator.join([s]*n)

print(concat('good', 5, ','))
```

■ 不推荐使用+运算符连接字符串，优先使用join()方法。

```
import timeit
#使用列表推导式生成10000个字符串
strlist = ['This is a long string that will not keep in memory.'
           for n in range(10000)]
#使用字符串对象的join()方法连接多个字符串
def use_join():
    return ''.join(strlist)
#使用运算符+连接多个字符串
def use_plus():
    result = ''
    for strtemp in strlist:
        result = result+strtemp
    return result
if __name__ == '__main__':
    times = 1000
    jointimer = timeit.Timer('use_join()', 'from __main__ import
use_join')
    print('time for join:', jointimer.timeit(number=times))
    plustimer = timeit.Timer('use_plus()', 'from __main__ import
use_plus')
    print('time for plus:', plustimer.timeit(number=times))
```

time for join: 0.10216611442409469
time for plus: 1.414725147403292

1.1 timeit.timeit()函数： 创建一个Timer实例，并运行代码进行计时，默认将代码执行一百万次。

基本语法：

```
timeit.timeit(stmt='pass', setup='pass', timer=<default timer>, number=1000000)
```

参数说明：

- **stmt**：传入需要测试时间的代码，可以直接传入代码表达式或单个变量，也可以传入函数。传入函数时要在函数名后面加上小括号，让函数执行，如 stmt = 'func()' 。
- **setup**：传入 stmt 的运行环境，如 stmt 中使用到的参数、变量，要导入的模块等，如 `setup = 'from __main__ import func' (__main__表示当前的文件)` 。可以写一行语句，也可以写多行语句，写多行语句时用分号隔开。。
- timer：是当前操作系统的基本时间单位，默认会根据当前运行环境的操作系统自动获取(源码中已经定义)，保持默认即可。
- number：要测试代码的运行次数，默认1000000(一百万)次，可以自己修改运行次数。

4.1 字符串

- timeit模块还支持下面代码演示的用法，从运行结果可以看出，当需要对大量数据进行类型转换时，内置函数map()可以提供非常高的效率。

  其中timeit（）方法也可以用于获取小代码片段的执行时间

```
>>> import timeit
>>> timeit.timeit('"-".join(str(n) for n in range(100))', number=10000)  #运行10000次
0.3063435900577929
>>> timeit.timeit('"-".join([str(n) for n in range(100)])', number=10000)
0.27191914957273866
>>> timeit.timeit('"-".join(map(str, range(100)))', number=10000)
0.2111951817165902 4
```

4.1 字符串

- lower()、upper()、capitalize()、title()、swapcase()

```
>>> s = "What is Your Name?"
>>> s.lower()                          #返回小写字符串
'what is your name?'
>>> s.upper()                          #返回大写字符串
'WHAT IS YOUR NAME?'
>>> s.capitalize()                     #字符串首字符大写
'What is your name?'
>>> s.title()                          #每个单词的首字母大写
'What Is Your Name?'
>>> s.swapcase()                       #大小写互换
'wHAT IS yOUR nAME?'
```

- 查找替换replace()，类似于Word中的“全部替换”功能。

```
>>> s = "中国，中国"
>>> s2 = s.replace("中国", "中华人民共和国")  #两个参数都作为一个整体
>>> print(s2)
中华人民共和国，中华人民共和国
```

4.1 字符串

- **应用：**测试用户输入中是否有敏感词，如果有的话就把敏感词替换为3个星号***。

```
words = ('测试', '非法', '暴力', '话')
text = '这句话里含有非法内容'
for word in words:
    if word in text:
        text = text.replace(word, '***')
print(text)
```



这句***里含有***内容

4.1 字符串

- 字符串对象的maketrans()方法用来生成字符映射表，而translate()方法用来根据映射表中定义的对应关系转换字符串并替换其中的字符，使用这两个方法的组合可以同时处理多个字符。

可应用于加密问题

这两个参数不是作为整体进行处理的

类似于生成一个密码本

```
#创建映射表，将字符"abcdef123"一一对应地转换为"uvwxyz@#$"
>>> table = ''.maketrans('abcdef123', 'uvwxyz@#$')
>>> s = "Python is a greate programming language. I like it!"
#按映射表进行替换
>>> s.translate(table)
'Python is u gryuty progrumming lunguugy. I liky it!'
```



4.1 字符串

- strip()、rstrip()、lstrip()　删除空白或指定字符

```
>>> '\n\nhello world   \n\n'.strip()          #删除空白字符
'hello world'
>>> "aaaassddafaaa".strip("a")                #删除左右边指定字符
'ssddaf'
>>> "aaaassddfaaa".rstrip("a")                #删除字符串右端指定字符
'aaaassddf'
>>> "aaaassddfaaa".lstrip("a")                #删除字符串左端指定字符
'ssddfaaa'
```

4.1.3 字符串常用方法



剥玉米

- 这三个函数的参数指定的字符串并不作为一个整体对待，
- 而是在原字符串的两侧、右侧、左侧删除参数字符串中
- 包含的所有字符，一层一层地从外往里扒。

```
>>> 'aabbccddeeeffg'.strip('af')  #字母f不在字符串两侧，所以不删除
'bbccddeeeffg'
>>> 'aabbccddeeeffg'.strip('gaf')
'bbccddeee'
>>> 'aabbccddeeeffg'.strip('gaef')
'bbccdd'
>>> 'aabbccddeeeffg'.strip('gbaef')
'ccdd'
>>> 'aabbccddeeeffg'.strip('gbaefcd')
''
```

```
>>> stra = "123abcrunoob321"
>>> stra.strip('21')
'3abcrunoob3'
```

从原字符串从外往里，一个一个字符，看是否包含在 strip('XXX')的 'XXX'中，如果包含，则剥掉！

✓ 文本规范化

```
>>> text = '''姓名：张三
年龄：39
性别男
职业    学生
籍贯：    地球'''
>>> infomation = text.split('\n')
>>> infomation
['姓名：张三', '年龄：39', '性别男', '职业    学生', '籍贯：    地球']
>>> for item in infomation:
    print(item[:2], item[2:].strip('：  '), sep='：')

姓名：张三
年龄：39
性别：男
职业：学生
籍贯：地球
```

第3个字符以后的所有字符，
如： 地球

4.1 字符串

- **Python字符串支持与整数的乘法运算，表示序列重复，也就是字符串内容的重复，得到新字符串。**

```
>>> 'abcd' * 3
'abcdabcdabcd'
```

- **s.startswith(t)、s.endswith(t)，判断字符串是否以指定字符串开始或结束**

```
>>> s = 'Beautiful is better than ugly.'
>>> s.startswith('Be')          #检测整个字符串
True
>>> s.startswith('Be', 5)       #指定检测范围起始位置
False
>>> s.startswith('Be', 0, 5)    #指定检测范围起始和结束位置
True
>>> import os
>>> [filename
      for filename in os.listdir(r'c:\\')
      if filename.endswith(('.bmp','.jpg','.gif'))]
```

4.1 字符串

- center()、ljust()、rjust()，返回指定宽度的新字符串，原字符串居中、左对齐或右对齐出现在新字符串中，如果指定宽度大于字符串长度，则使用指定的字符（默认为空格）进行填充。

```
>>> 'Hello world!'.center(20)          #居中对齐，以空格进行填充
'    Hello world!    '
>>> 'Hello world!'.center(20, '=')     #居中对齐，以字符=进行填充
'====Hello world!===='
>>> 'Hello world!'.ljust(20, '=')      #左对齐
'Hello world!========'
>>> 'Hello world!'.rjust(20, '=')      #右对齐
'========Hello world!'
```

- **zfill()返回指定宽度的字符串，在左侧以字符0进行填充。**

```
>>> 'abc'.zfill(5)          #在左侧填充数字字符0
'00abc'
>>> 'abc'.zfill(2)          #指定宽度小于字符串长度时，返回字符串本身
'abc'
>>> 'uio'.zfill(20)
'00000000000000000uio'
```

- **isalnum()、isalpha()、isdigit()、isdecimal()、isnumeric()、isspace()、isupper()、islower()，**用来测试字符串是否为数字或字母、是否为字母、是否为数字字符、是否为空白字符、是否为大写字母以及是否为小写字母。

```
>>> '1234abcd'.isalnum()
True
>>> '1234abcd'.isalpha()
False
>>> '1234abcd'.isdigit()
False
>>> 'abcd'.isalpha()
True
>>> '1234.0'.isdigit()
False
>>> '九'.isnumeric()                    #isnumeric()方法支持汉字数字
True
```

4.1 字符串

- 除了字符串对象提供的方法以外，很多Python内置函数也可以对字符串进行操作

```
>>> x = 'Hello world.'
>>> len(x)                          #字符串长度
12
>>> max(x)                          #最大字符
'w'
>>> min(x)
' '
>>> max(['abc', 'ABD'], key=str.upper) #忽略大小写
'ABD'
```

- 切片也适用于字符串，但仅限于读取其中的元素，不支持字符串修改。

```
>>> 'Explicit is better than implicit.'[:8]
'Explicit'
>>> 'Explicit is better than implicit.'[9:23]
'is better than'
```

内置函数eval()

eval函数尝试把任意字符串转化为python表达式并求值

```
>>> eval("3+4")
7
>>> a = 3
>>> b = 5
>>> eval('a+b')
8
>>> eval('aa')          #当前上下文中不存在对象aa
NameError: name 'aa' is not defined
>>> eval('*'.join(map(str, range(1, 6))))    #5的阶乘
120
```

4.1.4 与字符串处理相关的标准库

- Pytho标准库zlib中提供的compress()和decompress()函数可以用于字节串的压缩和解压缩。

```
>>> import zlib
>>> x = 'Python程序设计系列图书，董付国编著，清华大学出版社'.encode()
>>> len(x)
72
>>> y = zlib.compress(x)
>>> len(y)                                    #对于重复度比较小的信息，压缩比小
83
>>> x = ('Python系列图书'*3).encode()
>>> len(x)
54
>>> y = zlib.compress(x)
>>> len(y)
30
```

4.1.4 与字符串处理相关的标准库

- Python标准库string中定义数字字符、标点符号、英文字母、大写字母、小写字母等常量。

```
>>> import string
>>> string.digits     #数字符号常量
'0123456789'
>>> string.punctuation  #标点符号常量
'!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
>>> string.ascii_letters
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'
>>> string.ascii_lowercase
'abcdefghijklmnopqrstuvwxyz'
>>> string.ascii_uppercase
'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
```

✓ 应用：随机密码生成原理。

```
>>> import string
>>> characters = string.digits + string.ascii_letters
>>> import random
>>> ''.join([random.choice(characters) for i in range(8)])
'J5Cuofhy'
>>> ''.join([random.choice(characters) for i in range(16)])
'zSabpGltJ0X4CCjh'
```



4.1 字符串

- 在Python中，字符串属于不可变对象，不支持原地修改，如果需要修改其中的值，只能重新创建一个新的字符串对象。然而，如果确实需要一个支持原地修改的unicode数据对象，可以使用io.StringIO对象或array模块。

```
>>> import io
>>> s = “Hello, world”
>>> sio = io.StringIO(s)
>>> sio.getvalue()
'Hello, world'
>>> sio.seek(7)
7
>>> sio.write("there!")
6
>>> sio.getvalue()
'Hello, there!'
```

➢ 中英文分词

```
>>> import jieba                                    #导入jieba模块
>>> x = '分词的准确度直接影响了后续文本处理和挖掘算法的最终效果。'
>>> jieba.cut(x)                                    #使用默认词库进行分词
<generator object Tokenizer.cut at 0x000000000342C990>
>>> list(_)
['分词', '的', '准确度', '直接', '影响', '了', '后续', '文本处理', '
和', '挖掘', '算法', '的', '最终', '效果', '。']
>>> jieba.lcut('Python可以这样学，Python程序设计开发宝典') #直接给出
列表
Dumping model to file cache
C:\Users\d\AppData\Local\Temp\jieba.cache
['Python', '可以', '这样', '学', '，', 'Python', '程序设计', '开发',
'宝典']
>>> list(jieba.cut('花纸杯'))
['花', '纸杯']
>>> jieba.add_word('花纸杯')                        #增加词条
>>> list(jieba.cut('花纸杯'))                       #使用新题库进行分词
['花纸杯']
```

➢ 中英文分词

```
>>> from jieba import posseg
>>> text = '分词的准确度直接影响了后续文本处理和挖掘算法的最终效果。'
>>> for word, tag in posseg.cut(text):  # 得到分词及其词性
    print(word, tag, sep=':')
```

```
分词:n
的:uj
准确度:n
直接:ad
影响:vn
了:ul
后续:v
文本处理:n
和:c
挖掘:v
算法:n
的:uj
最终:d
效果:n
。:x
```

4.2 字符串的应用

中英文分词

```
>>> import snownlp                              #导入snownlp模块
>>> snownlp.SnowNLP('学而时习之，不亦说乎').words
['学而', '时习', '之', '，', '不亦', '说乎']
>>> snownlp.SnowNLP(x).words
['分词', '的', '准确度', '直接', '影响', '了', '后续', '文本', '处理', '和',
'挖掘', '算法', '的', '最终', '效果', '。’]
```

案例：分析《三国演义》小说中，人物出现最多次数的几个人。



4.2 字符串的应用

DMIC -HFUT-

- ✓ 编写程序，把一个英文句子中的单词倒置，标点符号不倒置，例如 I like python. 经过函数后变为：python. like I

```
def rev1(s):
    return ' '.join(reversed(s.split()))

def rev2(s):
    t = s.split()
    t.reverse()
    return ' '.join(t)

def rev5(s):
    '''字符串整体逆序，分隔，再各单词逆序'''
    t = ''.join(reversed(s)).split()
    t = map(lambda x:''.join(reversed(x)), t)
    return ' '.join(t)
```

4.2 字符串的应用

✓ 例：编写程序，查找一个字符串中最长的数字子串。

```
s = input("input string:")                  # s = "324fasafkda349349304230fsda"
result = []
t = []
for ch in s:                                # 遍历字符串中所有字符
    if '0'<=ch<='9':                        # 遇到数字，记录到临时变量
        t.append(ch)
    elif t:
        result.append(''.join(t)) #遇到非数字，把临时的连续数字记下来
        t = []
if t:                                       # 考虑原字符串以数字结束的情况
    result.append(''.join(t))

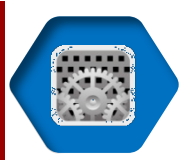
if result:
    print(max(result, key=len))             #len函数
else:
    print('No ')
```

```
349349304230
>>> result
['324', '349349304230']
```

```
# 选择法
def longest(s):
    length = len(s)
    start = 0
    span = (0, 0)
    for pos in range(length):
        if s[pos].isdigit() and (pos==0 or not s[pos-1].isdigit()):
            start = pos
        elif ((not s[pos].isdigit()) and s[pos-1].isdigit()
              and pos-start>span[1]-span[0]):
            span = (start, pos-1)
    #字符串以数字结束的情况
    if s[pos].isdigit() and pos-start>=span[1]-span[0]:
        span = (start, pos)
    return s[span[0]:span[1]+1]

ss = ('111abc2d3', 'abc111111d', 'a2bc11111111')
for s in ss:
    print(s, longest(s), sep=':')
```

```
111abc2d3:111
abc111111d:111111
a2bc11111111:11111111
>>>
```



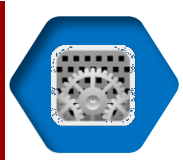
4.3 正则表达式

- 正则表达式使用某种预定义的模式去匹配一类具有共同特征的字符串，
  - 主要用于处理字符串，可以快速、准确地完成复杂的查找、替换等处理要求
  - 在文本编辑与处理、网页爬虫之类的场合中有重要应用。
- Python中，re模块提供了正则表达式操作所需要的功能。

4.3 正则表达式

4.3.1 正则表达式语法

元字符	功能说明
.	匹配除换行符以外的任意单个字符
*	匹配位于*之前的字符或子模式的0次或多次出现
+	匹配位于+之前的字符或子模式的1次或多次出现，即至少有一次
-	在[]之内用来表示范围
|	匹配位于|之前或之后的字符
^	匹配行首，匹配以^后面的字符开头的字符串
$	匹配行尾，匹配以$之前的字符结束的字符串
?	匹配位于?之前的0个或1个字符。当此字符紧随任何其他限定符（*、+、?、{n}、{n,}、{n,m}）之后时，匹配模式是“非贪心的”。“非贪心的”模式匹配搜索到的、尽可能短的字符串，而默认的“贪心的”模式匹配搜索到的、尽可能长的字符串。例如，在字符串“oooo”中，“o+?”只匹配单个“o”，而“o+”匹配所有“o”
\	表示位于\之后的为转义字符
\num	此处的num是一个正整数，表示子模式编号。 例如，“(.)\1”匹配两个连续的相同字符
\f	换页符匹配
\n	换行符匹配



4.3 正则表达式

元字符	功能说明
\r	匹配一个回车符
\b	匹配单词头或单词尾
\B	与\b含义相反
\d	匹配任何数字，相当于[0-9]
\D	与\d含义相反，等效于[^0-9]
\s	匹配任何空白字符，包括空格、制表符、换页符，与 [\f\n\r\t\v] 等效
\S	与\s含义相反
\w	匹配任何字母、数字以及下划线，相当于[a-zA-Z0-9_]
\W	与\w含义相反\w含义相反，与“[^A-Za-z0-9_]”等效
()	将位于()内的内容作为一个整体来对待
{m,n}	{}前的字符或子模式重复至少m次，至多n次
[]	表示范围，匹配位于[]中的任意一个字符
[^xyz]	反向字符集，匹配除x、y、z之外的任何字符
[a-z]	字符范围，匹配指定范围内的任何字符
[^a-z]	反向范围字符，匹配除小写英文字母之外的任何字符

\b 匹配单词边界，指单词和空格的位置

```
>>> re.search(r"\babc\b","abc abcd")
<re.Match object; span=(0, 3), match='abc'>
```

'*'表示匹配前一个字符重复 0 次到无限次，'+'表示匹配前一个字符重复 1次到无限次，'?'表示匹配前一个字符重复 0 次到1次（贪婪匹配，也就是尽可能多匹配）

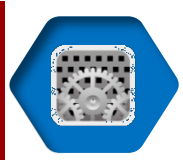
```
>>> re.search(r'abc*',"abcccd")
<re.Match object; span=(0, 5), match='abccc'>
>>> re.search(r'abc?',"abcccd")
<re.Match object; span=(0, 3), match='abc'>
```

后面加?号使其变成惰性匹配即非贪婪匹配

```
>>> re.search(r'abc*?',"abcccd")
<re.Match object; span=(0, 2), match='ab'>
>>> re.search(r'abc??',"abcccd")
<re.Match object; span=(0, 2), match='ab'>
```

.一个点只能匹配一个字符

```
>>> re.search(r'a.',"D567Dabc")
<re.Match object; span=(5, 7), match='ab'>
```



4.3 正则表达式

\d 匹配任何十进制数，相当于[0-9]

例： r"\d+" 正则表达式表示匹配连续的多个数值，"abc123cd"字符串中搜索连续的数值,得到"123"

```
>>> import re
>>> re.search(r"\d+","abc123cd")
<re.Match object; span=(3, 6), match='123'>
```

例：r"b\d+" 第一个字符要匹配"b",后面是连续的多个数字，如 b1234

例："$"匹配行尾，匹配以$之前的字符结束的字符串

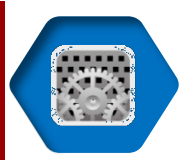
```
>>> re.search(r"ab$","abcabab")
<re.Match object; span=(5, 7), match='ab'>
```

例："^"匹配行首，匹配以^后面的字符开头的字符串

```
>>> re.search(r"^ab","abcabab")
<re.Match object; span=(0, 2), match='ab'>
```

例：(…)为一个整体,经常与"+"、"*"、"?"的连续使用,对（…）部分进行重复.

```
>>> re.search(r"^(ab)+","ababcad")
<re.Match object; span=(0, 4), match='abab'>
```

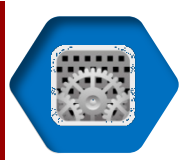


4.3 正则表达式

\1 表示对前面第一个捕获分组内容的引用。例如
([A-Z])567\1表示匹配前后为相同大写字母包围567的字串。

```
>>> re.search(r'([A-Z])567\1',"D567Dabc")
<re.Match object; span=(0, 5), match='D567D'>
```

如果\1前面没有捕获的分组即用()括起来的匹配，则\1表示匹配八进制数字1

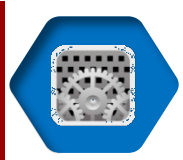


4.3 正则表达式

- 最简单的正则表达式是普通字符串，可以匹配自身
- '[pjc]ython'可以匹配'python'、'jython'、'cython'
- '[a-zA-Z0-9]'可以匹配一个任意大小写字母或数字
- '[^abc]'可以一个匹配任意除'a'、'b'、'c'之外的字符
- 'python|perl'或'p(ython|erl)'都可以匹配'python'或'perl'
- 子模式后面加上问号表示可选。r'(http://)?(www\.)?python\.org'

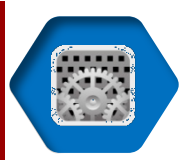
只能匹配'http://www.python.org'、'http://python.org'、'www.python.org'和'python.org'

- '^http'只能匹配所有以'http'开头的字符串
- (pattern)*：允许模式重复0次或多次
- (pattern)+：允许模式重复1次或多次
- (pattern){m,n}：允许模式重复m~n次



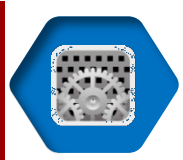
4.3 正则表达式

- '(a|b)*c'：匹配多个（包含0个）a或b，后面紧跟一个字母c。
- 'ab{1,}'：等价于'ab+'，匹配以字母a开头后面带1个至多个字母b的字符串。
- '^[a-zA-Z]{1}([a-zA-Z0-9._]){4,19}$'：匹配长度为5-20的字符串，必须以字母开头并且可带字母、数字、“_”、“.”的字符串。
- '^(\w){6,20}$'：匹配长度为6-20的字符串，可以包含字母、数字、下划线。
- '^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$'：检查给定字符串是否为合法IP地址。
- r'^(13[0-9]|15[012356789]|17[678]|18[0-9]|14[57])[0-9]{8}$'：检查给定字符串是否为手机号码。
- '^[a-zA-Z]+$'：检查给定字符串是否只包含英文字母大小写。
- '^\w+@(\w+\.)+\w+$'：检查给定字符串是否为合法电子邮件地址。



4.3 正则表达式

- '^(\-)?\d+(\.\d{1,2})?$'：检查给定字符串是否为最多带有2位小数的正数或负数。
- '[\u4e00-\u9fa5]'：匹配给定字符串中所有汉字。
  unicode的中文编码表，第一个“4e00”最后一个“9fa0”，总共有20901个汉字
- '^\d{18}|\d{15}$'：检查给定字符串是否为合法身份证格式。
- '\d{4}-\d{1,2}-\d{1,2}'：匹配指定格式的日期，例如2016-1-31。
- ‘(.)\\1+’或 r'(.)\1+'： 匹配任意字符的两次或多次重复出现。



4.3 正则表达式

➢ 4.3.2 re模块的主要方法与使用

方法	功能说明
compile(pattern[, flags])	创建模式对象
escape(string)	将字符串中所有特殊正则表达式字符转义
findall(pattern, string[, flags])	返回包含字符串中所有与给定模式匹配的项的列表
finditer(pattern, string, flags=0)	返回包含所有匹配项的迭代对象，其中每个匹配项都是match对象
fullmatch(pattern, string, flags=0	尝试把模式作用于整个字符串，返回match对象或None
match(pattern, string[, flags])	从字符串的开始处匹配模式，返回match对象或None
purge()	清空正则表达式缓存
search(pattern, string[, flags])	在整个字符串中寻找模式，返回match对象或None
split(pattern, string[, maxsplit=0])	根据模式匹配项分隔字符串
sub(pat, repl, string[, count=0])	将字符串中所有与pat匹配的项用repl替换，返回新字符串，repl可以是字符串或返回字符串的可调用对象，作用于每个匹配的match对象
subn(pat, repl, string[, count=0])	将字符串中所有pat的匹配项用repl替换，返回包含新字符串和替换次数的二元元组，repl可以是字符串或返回字符串的可调用对象，作用于每个匹配的match对象

4.3 正则表达式

4.3.2 re模块的主要方法与使用

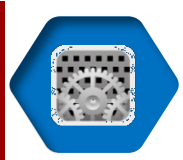
```
>>> import re                                    #导入re模块
>>> text = 'alpha. beta....gamma delta'          #测试用的字符串
>>> re.split('[\. ]+', text)          #使用指定字符（.与空格）作为分隔符进行分隔
['alpha', 'beta', 'gamma', 'delta']
>>> re.split('[\. ]+', text, maxsplit=2) #最多分隔2次
['alpha', 'beta', 'gamma delta']
>>> re.split('[\. ]+', text, maxsplit=1) #最多分隔1次
['alpha', 'beta....gamma delta']
>>> pat = '[a-zA-Z]+'
>>> re.findall(pat, text)                        #查找所有单词
['alpha', 'beta', 'gamma', 'delta']
```

- 把单词中间误写作大写的字母改为小写：

```
import re
def checkModify(s):
    return re.sub(r'\b(\w)(\w+)(\w)\b',
                  lambda x: x.group(1)+x.group(2).lower()+x.group(3),s)
print(checkModify('aBc ABBC D eeee fFFFfF'))
```

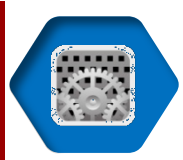
group（）在正则表达式中用于获取分段截获的字符串

abc AbbC D eeee fffffF



4.3 正则表达式

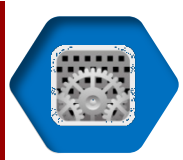
```
>>> pat = '{name}'
>>> text = 'Dear {name}...'
>>> re.sub(pat, 'Mr.Python', text)          #字符串替换
'Dear Mr.Python...'
>>> s = 'a s d'
>>> re.sub('a|s|d', 'good', s)              #字符串替换
'good good good'
>>> s = "It's a very good good idea"
>>> re.sub(r'(\b\w+) \1', r'\1', s)         #处理连续的重复单词
"It's a very good idea"
>>> re.sub(r"((\w+) )\1", r"\2", s) #\2匹配第二个()匹配到的内容，即(\w+)
#\1匹配第一个()匹配到的内容，即((\w+) )
"It's a very goodidea"
>>> re.sub('a', lambda x:x.group(0).upper(),
           'aaa abc abde')                  #repl为可调用对象
'AAA Abc Abde'
```



4.3 正则表达式

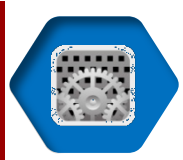
```
>>> re.sub('[a-z]', lambda x:x.group(0).upper(), 'aaa abc abde')
'AAA ABC ABDE'
>>> re.sub('[a-zA-z]', lambda x:chr(ord(x.group(0))^32),
           'aaa aBc abde')                    #英文字母大小写互换
'AAA AbC ABDE'
>>> re.subn('a', 'dfg', 'aaa abc abde') #返回新字符串和替换次数
('dfgdfgdfg dfgbc dfgbde', 5)
>>> re.sub('a', 'dfg', 'aaa abc abde')
'dfgdfgdfg dfgbc dfgbde'
>>> re.escape('http://www.python.org')  #字符串转义
'http://www\\.python\\.org'
>>> from string import punctuation
>>> re.sub('['+punctuation+']', '', 'abcd,.e!f/?') #删除标点符号
'abcdef'
```

punctuation其中定义的是所有英文的标点符号，‘[’+punctuation+‘]’类似于[a-z]，表示标点符号中的任意一个符号，替换字符‘’中间没有空格



4.3 正则表达式

```
>>> example = 'Beautiful is better than ugly.'
>>> re.findall('\\bb.+?\\b', example)  #以字母b开头的完整单词
                                       #此处问号?表示非贪心模式
['better']
>>> re.findall('\\bb.+\\b', example)   #贪心模式的匹配结果
['better than ugly']
>>> re.findall('\\bb\w*\\b', example)
['better']
>>> re.findall('\\Bh.+?\\b', example)
                                  #不以h开头且含有h字母的单词剩余部分
['han']
>>> re.findall('\\b\w.+?\\b', example) #所有单词
['Beautiful', 'is', 'better', 'than', 'ugly']
```



4.3 正则表达式

```
>>> re.findall('\d+\.\d+\.\d+', 'Python 2.7.13')
                                        #查找并返回x.x.x形式的数字
['2.7.13']
>>> re.findall('\d+\.\d+\.\d+', 'Python 2.7.13,Python 3.6.0')
['2.7.13', '3.6.0']
>>> s = '<html><head>This is head.</head><body>This is
body.</body></html>'
>>> pattern = r'<html><head>(.+)</head><body>(.+)</body></html>'
>>> result = re.search(pattern, s)
>>> result.group(1)                     #第一个子模式
'This is head.'
>>> result.group(2)                     #第二个子模式
'This is body.'
```

4.3.2 re模块的主要方法与使用

✓ 应用：将一句话的单词进行倒置，标点不倒置。比如 I like python. 经过函数后变为：python. like I

```
import re
 s = "I like Python."
 t = re.split('\s+', s.strip()) # '''考虑开头或结束有空格的情况'''
 t.reverse()
 print(' '.join(t))

 import re
 s = "I like Python."
 t = re.split('\s+', s.strip())
 print(' '.join(reversed(t)))
```

4.3 正则表达式

- ✓ 应用：查找字符串中最长的数字子串

```
import re
s = input("str:")
t = re.findall('\d+', s)
if t:
    print(max(t, key=len))
print('No ')
```

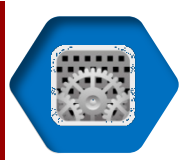
```
'''使用非数字作为分隔符'''
import re
s = input("str:")
t = re.split('[^\d]+', s)
if t:
    print(max(t, key=len))
print('No ')
```

re.findall():函数返回包含所有匹配项的列表
re.split():返回一个列表,其中字符串在每次匹配时被拆分。

4.3.3 使用正则表达式对象

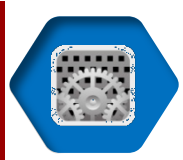
- 首先使用re模块的compile()方法将正则表达式编译生成正则表达式对象，然后再使用正则表达式对象提供的方法进行字符串处理。

- 使用编译后的正则表达式对象可以提高字符串处理速度，也提供了更强大的文本处理功能。



4.3 正则表达式

- match()、search()、findall()
  - ✓ match(string[, pos[, endpos]])方法在字符串开头或指定位置进行搜索，模式必须出现在字符串开头或指定位置；
  - ✓ search(string[, pos[, endpos]])方法在整个字符串或指定范围中进行搜索；
  - ✓ findall(string[, pos[, endpos]])方法在字符串指定范围中查找所有符合正则表达式的字符串并以列表形式返回。



4.3 正则表达式

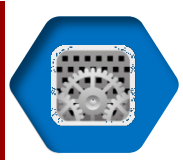
re.match 尝试从字符串的起始位置匹配一个模式

```
import re
s='pythonisbeautifulpython'
result=re.match('python',s)
print(result)
```

```
<re.Match object; span=(0, 6), match='python'>
(0, 6)
>>>
```

改为 s='2pythonisbeautiful'

如果不是起始位置匹配成功的话，match()返回的是空 none，



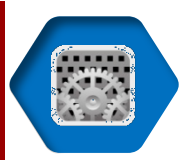
4.3 正则表达式

re.search匹配整个字符串，直到找到一个匹配。

注意：search也只能匹配到一个，找到符合规则的就返回，不会一直往后找

```
import re
s='2pythonisbeautifulpython'
result=re.search('python',s)
print(result)
print(result.span())
```

```
<re.Match object; span=(1, 7), match='python'>
(1, 7)
>>>
```

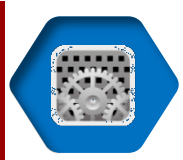


4.3 正则表达式

findall方法在字符串中找到正则表达式所匹配的所有子串，并返回一个列表，如果没有找到匹配的，则返回空列表

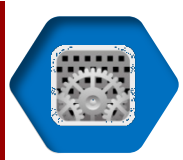
```
import re
s='2pythonisbeautifulpython'
result=re.findall('python',s)
print(result)
```

```
['python', 'python']
>>>
```



4.3 正则表达式

```
>>> import re
>>> example = 'Hefei University of Technology'
>>> pattern = re.compile(r‘\bH\w+\b’)  #H开头
>>> pattern.findall(example)
[‘Hefei']
>>> pattern = re.compile(r‘\w+y\b’)   #y结尾
>>> pattern.findall(example)
[‘University‘,’Technology’]
>>> pattern = re.compile(r'\b[a-zA-Z]{2}\b') #查找2个字母长的单词
>>> pattern.findall(example)
[‘of']
>>> s = 'ab134ab98723jafjweoruiagab'
>>> m = re.search(r'((ab).*){2}.*(ab)', s)#在s中查找ab的第3次出现
>>> m.group(3)
'ab'
>>> m.span(3)
(24, 26)
>>> s[24:]
'ab'
```

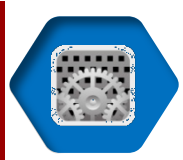


4.3 正则表达式

匹配账号:只能由字母和数字组成,长度为10位

```
import re
s='abc1234567efg'
result=re.match('[0-9a-zA-Z]{10}',s)
print(result)
```

```
>>> import re
>>> example = 'Hefei University of Technology'
>>> pattern = re.compile(r'\b\w*i\w*\b') #查找所有含有字母i的单词
>>> pattern.findall(example)
['‘Hefei', ‘University']
>>> text = "He was carefully disguised but captured quickly by police."
>>> re.findall(r"\w+ly", text)
['carefully', 'quickly']
```



4.3 正则表达式

- sub()、subn()

✓ 正则表达式对象的**sub(repl, string[, count = 0])**和**subn(repl, string[, count = 0])**方法用来实现字符串替换功能，其中参数**repl**可以为字符串或返回字符串的可调用对象。

```
>>> example = '''Beautiful is
better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.'''
```

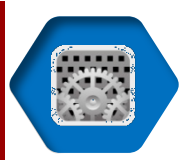
```
>>> pattern = re.compile(r'\bb\w*\b', re.I) #匹配以b或B开头的单词
>>> print(pattern.sub('*', example))        #将符合条件的单词替换为*

* is * than ugly.
Explicit is * than implicit.
Simple is * than complex.
Complex is * than complicated.
Flat is * than nested.
Sparse is * than dense.
Readability counts.
```

忽略大小写

```
>>> print(pattern.sub(lambda x: x.group(0).upper(), example))
                                        #把所有匹配项都改为大写

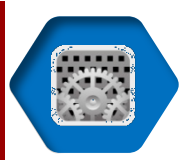
BEAUTIFUL is BETTER than ugly.
Explicit is BETTER than implicit.
Simple is BETTER than complex.
Complex is BETTER than complicated.
Flat is BETTER than nested.
Sparse is BETTER than dense.
Readability counts.
```



4.3 正则表达式

```
>>> print(pattern.sub('*', example, 1))        #只替换1次

* is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
```



4.3 正则表达式

- 正则表达式对象的split(string[, maxsplit = 0])方法用来实现字符串分隔

```
>>> example = r'one,two,three.four/five\six?seven[eight]nine|ten'
>>> pattern = re.compile(r'[,./\\?[\]\|]')
>>> pattern.split(example)
['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine', 'ten']
>>> example = r'one1two2three3four4five5six6seven7eight8nine9ten'
>>> pattern = re.compile(r'\d+')
>>> pattern.split(example)
['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine', 'ten']
>>> example = r'one two    three  four,five.six.seven,eight,nine9ten'
>>> pattern = re.compile(r'[\s,.\d]+')          #允许分隔符重复
>>> pattern.split(example)
['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine', 'ten']
```

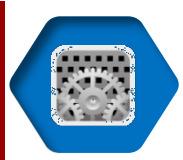
4.3.4 子模式与match对象

- 使用()表示一个子模式，括号中的内容作为一个整体处理
  - 例：‘(red)+’可以匹配‘redred’、‘redredred’等多个重复‘red’的情况

```
>>> telNumber = '''Suppose my Phone No. is 0551-1234567,
yours is 010-12345678, his is 025-87654321.'''
>>> pattern = re.compile(r'(\d{3,4})-(\d{7,8})')
>>> pattern.findall(telNumber)
[('0535', '1234567'), ('010', '12345678'), ('025', '87654321')]
```

4.3.4 子模式与match对象

- **子模式扩展语法：**
  - (?P<groupname>)：为子模式命名
  - (?iLmsux)：设置匹配标志，可以是几个字母的组合，每个字母含义与编译标志相同
  - (?:...)：匹配但不捕获该匹配的子表达式
  - (?P=groupname)：表示在此之前的命名为groupname的子模式
  - (?#...)：表示注释
  - (?=…)：用于正则表达式之后，表示如果=后的内容在字符串中出现则匹配，但不返回=之后的内容
  - (?!...)：用于正则表达式之后，表示如果!后的内容在字符串中不出现则匹配，但不返回!之后的内容
  - (?<=…)：用于正则表达式之前，与(?=…)含义相同
  - (?<!...)：用于正则表达式之前，与(?!...)含义相同



4.3 正则表达式

(?!...)：用于正则表达式之后，表示如果!后的内容在字符串中不出现则匹配，但不返回!之后的内容

例： r'(\w)(?!.*\1)'：查找字符串中每个字符的最后一次出现。

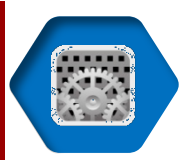
分析：(\w)表示第一次出现的模式，即任意一个字符，(?!.*\1)中，\1表示前面出现(\w)，即后面出现一个相同的字符，而 ?!表示后面不出现右侧模式中匹配的情况，而右侧括号里的正则是.*\1，其中.*表示任意的字符，即表示任意的字符后面紧跟着前面出现的字符，不出这种模式，即表示(\w) 为匹配字符串模式中，最后一次出现的字符

```
>>> re.search(r'(\w)(?!.*\1)',"bcbcad")
<re.Match object; span=(2, 3), match='b'>
```

(?=…)：用于正则表达式之后，表示如果=后的内容在字符串中出现则匹配，但不返回=之后的内容

例：r'(\w)(?=.*\1)'：查找字符串中所有重复出现的字符

```
>>> re.search(r'(\w)(?=.*\1)',"gfcbcdcd")
<re.Match object; span=(2, 3), match='c'>
```



4.3 正则表达式

(?<=[pattern])和(?=[pattern])是匹配到了[pattern]不会返回、亦不会记录（捕获）[pattern]子模式，

(?<=…)：用于正则表达式之前，与(?=…)含义相同

```
s='cake is better than potato'
pattern2=re.compile(r'(?<=is\s)better(\sthan)' )
print(pattern2.search(s))
```

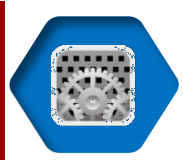
```
<re.Match object; span=(8, 19), match='better than'>
```

```
>>> pattern2=re.compile(r'(is\s)better(?=\sthan)')
>>> print(pattern2.search(s))
<re.Match object; span=(5, 14), match='is better'>
```

(?:[pattern])是匹配到了[pattern]会返回，但不会记录（捕获）[pattern]子模式，

```
>>> pattern2=re.compile(r'(?:is\s)better(\sthan)')
>>> print(pattern2.search(s))
<re.Match object; span=(5, 19), match='is better than'>
>>> print(pattern2.search(s).group(1))
 than
```

不会记录（捕获）

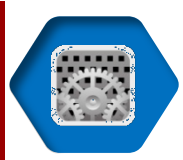


4.3 正则表达式

- r"\d+(?=[a-z]+)"：匹配连续的数字并且最后一个数字跟着小写字母。
- r"\d+(?![a-z]+)"：匹配连续的数字，并且最后一个数字后面不能跟小写字母。
- r"(?<=[a-z])\d+"：匹配连续的数字，并且第一个数字的前面是小写字母。
- r"(?<![a-z])\d+"：连续的数字，并且第一个数字的前面不能是小写字母。
- r'\d{3}(?!\d)'：匹配三位数字，而且这三位数字的后面不能是数字。
- r'\b((?!abc)\w)+\b'：匹配不包含连续字符串abc的单词。
- r'(?<![a-z])\d{7}'：匹配前面不是小写字母的七位数字。
- r"(?<=<(\w{4})>)(.*)(?=<\/\1>)"：匹配" hello world "中的span和hello world。

4.3 正则表达式

- r'(\w)(?!.*\1)'：查找字符串中每个字符的最后一次出现。
- r'(\w)(?=.*\1)'：查找字符串中所有重复出现的字符。
- '^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[,._]).{8,}$'：检查给定字符串是否为强密码，必须同时包含英语字母大写字母、英文小写字母、数字或特殊符号（如英文逗号、英文句号、下划线），并且长度必须至少8位。
- "(?!.*[\'\"\/;=%?]).+"：如果给定字符串中包含'、"、/、;、=、%、?则匹配失败。
- '((?P<f>\b\w+\b)\s+(?P=f))'：匹配连续出现两次的单词。
- '((?P<f>.)(?P=f)(?P<g>.)(?P=g))'：匹配AABB形式的成语或字母组合。



4.3 正则表达式

- 正则表达式对象的match方法和search方法匹配成功后返回match对象。
- match对象的主要方法有：

✓ group()：返回匹配的一个或多个子模式内容

✓ groups()：返回一个包含匹配的所有子模式内容的元组

✓ groupdict()：返回包含匹配的所有命名子模式内容的字典

✓ start()：返回指定子模式内容的起始位置

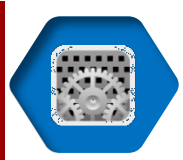
✓ end()：返回指定子模式内容的结束位置的前一个位置

✓ span()：返回一个包含指定子模式内容起始位置和结束位置前一个位置的元组。

4.3.4 子模式与match对象

- match对象的用法

```
>>> m = re.match(r"(\w+) (\w+)", "Isaac Newton, physicist")
>>> m.group(0)                    #返回整个模式内容
'Isaac Newton'
>>> m.group(1)                    #返回第1个子模式内容
'Isaac'
>>> m.group(2)                    #返回第2个子模式内容.
'Newton'
>>> m.groups()                    #返回一个包含匹配的所有子模式内容的元组
('Isaac', 'Newton')
```



4.3 正则表达式

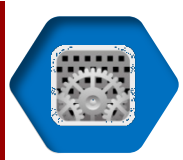
```
>>> exampleString = '''There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than right now.'''
>>> pattern = re.compile(r'(?<=\w\s)never')     #查找不是第一个单词的never
>>> matchResult = pattern.search(exampleString)
>>> matchResult.span()
(156, 161)
>>> pattern = re.compile(r'(?:is\s)better(\sthan)')
                                                #查找前面是is的better than组合
>>> matchResult = pattern.search(exampleString)
>>> print(matchResult)
<re.Match object; span=(141, 155), match='is better than’>
>>> matchResult.group(0)                        #组0表示整个模式
'is better than'
>>> matchResult.group(1)
' than'
```

```
>>> m = re.match(r"(?P<first_name>\w+) (?P<last_name>\w+)", "Malcolm Reynolds")
>>> m.group('first_name')      #使用命名的子模式
'Malcolm'
>>> m.group('last_name')
'Reynolds'
>>> m = re.match(r"(\d+)\.(\d+)", "24.1632")
>>> m.groups()                 #返回所有匹配的子模式（不包括第0个）
('24', '1632')
>>> m = re.match(r"(?P<first_name>\w+) (?P<last_name>\w+)", "Malcolm Reynolds")
>>> m.groupdict()              #以字典形式返回匹配的结果
{'first_name': 'Malcolm', 'last_name': 'Reynolds'}
>>> exampleString = '''There should be one-- and preferably only one --obvious way to do it.Although that way may not be obvious at first unless you're Dutch.
Now is better than never.Although never is often better than right now.'''
>>> pattern = re.compile(r'(?<=\w\s)never(?=\s\w)') #查找不在句子开头和结尾的never
>>> matchResult = pattern.search(exampleString)
>>> matchResult.span()
(172, 177)
```

4.3.4 子模式与match对象

```
                                          #查找前面没有单词not的单词be
>>> pattern = re.compile(r'(?<!not\s)be\b')
>>> index = 0
>>> while True:
    matchResult = pattern.search(exampleString, index)
    if not matchResult:
        break
    print(matchResult.group(0), ':', matchResult.span(0))
    index = matchResult.end(0)

be : (13, 15)
>>> exampleString[13:20]
'be one-'
```



4.3 正则表达式

➢ 4.3.4 子模式与match对象

正则表达式的模式修饰符

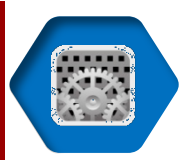
表示匹配时不区分大小写

```
>>> pattern = re.compile(r'\b(?i)n\w+\b')
>>> index = 0
>>> while True:
    matchResult = pattern.search(exampleString, index)
    if not matchResult:
        break
    print(matchResult.group(0), ':', matchResult.span(0))
    index = matchResult.end(0)

not : (92, 95)
Now : (137, 140)
never : (156, 161)
never : (172, 177)
now : (205, 208)
```

4.3.4 子模式与match对象

```
>>> pattern = re.compile(r'(\b\w*(?P<f>\w+)(?P=f)\w*\b)')
>>> index = 0                                    #有连续相同字母（1个或多个）的单词
>>> while True:
    matchResult = pattern.search(exampleString, index)
    if not matchResult:
        break
    print(matchResult.group(0), ':', matchResult.group(2))
    index = matchResult.end(0) + 1
unless : s
better : t
better : t
>>> s = 'aabc abcd abcabcabcd’
>>> pattern.findall(s)
[('aabc', 'a'), ('abcabcabcd', 'abc')]
```



4.3 正则表达式

- 应用：使用正则表达式提取字符串中的电话号码（方法一）

```
import re
telNumber = '''Suppose my Phone No. is 0551-1234567, yours is 010-12345678,
his is 025-87654321.'''
pattern = re.compile(r'(\d{3,4})-(\d{7,8})')
index = 0
while True:
    matchResult = pattern.search(telNumber, index)  #从指定位置开始匹配
    if not matchResult:
        break
    print('-'*30)
    print('Success:')
    for i in range(3):  #3个电话号码
        print('Searched content:', matchResult.group(i), ' Start from:',
              matchResult.start(i), 'End at:', matchResult.end(i),
              ' Its span is:', matchResult.span(i))
    index = matchResult.end(2)                  #指定下次匹配的开始位置
```

```
------------------------------
Success:
Searched content: 0551-1234567  Start from: 24 End at: 36  Its span is: (24, 36)
Searched content: 0551  Start from: 24 End at: 28  Its span is: (24, 28)
Searched content: 1234567  Start from: 29 End at: 36  Its span is: (29, 36)
------------------------------
Success:
Searched content: 010-12345678  Start from: 47 End at: 59  Its span is: (47, 59)
Searched content: 010  Start from: 47 End at: 50  Its span is: (47, 50)
Searched content: 12345678  Start from: 51 End at: 59  Its span is: (51, 59)
------------------------------
Success:
Searched content: 025-87654321  Start from: 68 End at: 80  Its span is: (68, 80)
Searched content: 025  Start from: 68 End at: 71  Its span is: (68, 71)
Searched content: 87654321  Start from: 72 End at: 80  Its span is: (72, 80)
>>>
```

4.3 正则表达式

- ✓ 应用：使用正则表达式提取字符串中的电话号码（方法二）

```
import re

text = '''Suppose my Phone No. is 0551-1234567,
yours is 010-12345678,
his is 025-87654321.'''
matchResult = re.findall(r'(\d{3,4})-(\d{7,8})', text)
for item in matchResult:
    print(item[0], item[1], sep='-')
```

注意：findall返回一个列表

```
0551-1234567
010-12345678
025-87654321
```

```
>>> print(matchResult)
[('0551', '1234567'), ('010', '12345678'), ('025', '87654321')]
>>>
```

4.4 正则表达式的应用

- 正则表达式的应用案例
  - ✓ 例：使用正则表达式批量检查网页文件是否被嵌入iframe框架。

```
import os
import re

def detectIframe(fn):
    #存放网页文件内容的列表
    content = []
    with open(fn, encoding='utf8') as fp:
        #读取文件所有行，删除两侧的空白字符，然后添加到列表中
        for line in fp:
            content.append(line.strip())
    #把所有内容连接成字符串
    content = ' '.join(content)
    #正则表达式
    m = re.findall(r'<iframe\s+src=.*?></iframe>', content)
    if m:
        #返回文件名和被嵌入的框架
        return {fn:m}
    return False
```

添加 iframe 的语法

```
<iframe src="URL"></iframe>
```

```
#遍历当前文件夹中所有html和htm文件并检查是否被嵌入框架
for fn in (f for f in os.listdir('.') if f.endswith(('.html','.htm'))):
    r = detectIframe(fn)
    if not r:
        continue
    #输出检查结果
    for k, v in r.items():
        print(k)
        for vv in v:
            print('\t', vv)
```



4.4 正则表达式的应用

- **例：查找字符串中每个字符的首次出现：**给定一个任意字符串，要求得到一个新字符串，重复字符只保留一个，并且新字符串中的字符保持在原字符串中首次出现的先后顺序。
- ✓ 例如，abcdaaabe处理后应得到abcde。

```
from re import findall
from random import choice
from string import digits
text = ''.join(choice(digits) for _ in range(30))
def func1(text):
    result = set(text)
    return ''.join(sorted(result, key=lambda ch:text.index(ch)))

def func2(text):
    result = []
    for ch in text:
        if ch not in result:
            result.append(ch)
    return ''.join(result)
```

```
def func3(text):
    return ''.join(findall(r'(\w)(?!.*\1)', text[::-1]))[::-1]

print(text)
print(func1(text))
print(func2(text))
print(func3(text))
```

```
5124721394894845142248366800l0
5124739860
5124739860
5124739860
>>>
```

4.5 本章小结

内容回顾

- 字符串的相关概念：格式化方法、常用方法
- 正则表达式的规则

思考

- 正则表达式的应用